An Introduction to Computing with Neural Nets

Richard P. Lippmann

Abstract

Artificial neural net models have been studied for many years in the hope of achieving human-like performance in the fields of speech and image recognition. These models are composed of many nonlinear computational elements operating in parallel and arranged in patterns reminiscent of biological neural nets. Computational elements or nodes are connected via weights that are typically adapted during use to improve performance. There has been a recent resurgence in the field of artificial neural nets caused by new net topologies and algorithms, analog VLSI implementation techniques, and the belief that massive parallelism is essential for high performance speech and image recognition. This paper provides an introduction to the field of artificial neural nets by reviewing six important neural net models that can be used for pattern classification. These nets are highly parallel building blocks that illustrate neural- net components and design principles and can be used to construct more complex systems. In addition to describing these nets, a major emphasis is placed on exploring how some existing classification and clustering algorithms can be performed using simple neuron-like components. Single-layer nets can implement algorithms required by Gaussian maximum-likelihood classifiers and optimum minimum-error classifiers for binary patterns corrupted by noise. More generally, the decision regions required by any classification algorithm can be generated in a straightforward manner by three-layer feed-forward nets.

INTRODUCTION

Artificial neural net models or simply "neural nets" go by many names such as connectionist models, parallel distributed processing models, and neuromorphic systems. Whatever the name, all these models attempt to achieve good performance via dense interconnection of simple computational elements. In this respect, artificial neural net structure is based on our present understanding of biological nervous systems. Neural net models have greatest potential in areas such as speech and image recognition where many hypotheses are pursued in parallel, high computation rates are required, and the current best systems are far from equaling human performance. Instead of performing a program of instructions sequentially as in a von Neumann computer, neural net models explore many competing hypotheses simultaneously using massively parallel nets composed of many computational elements connected by links with variable weights.

Computational elements or nodes used in neural net models are nonlinear, are typically analog, and may be slow compared to modern digital circuitry. The simplest node sums N weighted inputs and passes the result through a nonlinearity as shown in Fig. 1. The node is characterized by an internal threshold or offset θ and by the type of nonlinearity. Figure 1 illustrates three common types of nonlinearities; hard limiters, threshold logic elements, and sigmoidal nonlinearities. More complex nodes may include temporal integration or other types of time dependencies and more complex mathematical operations than summation.

Neural net models are specified by the net topology, node characteristics, and training or learning rules. These rules specify an initial set of weights and indicate how weights should be adapted during use to improve performance. Both design procedures and training rules are the topic of much current research.

The potential benefits of neural nets extend beyond the high computation rates provided by massive parallelism. Neural nets typically provide a greater degree of robustness or fault tolerance than von Neumann sequential computers because there are many more processing nodes, each with primarily local connections. Damage to a few nodes or links thus need not impair overall performance significantly. Most neural net algorithms also adapt connection weights in time to improve performance based on current results. Adaptation or learning is a major focus of neural net research. The ability to adapt and continue learning is essential in areas such as speech recognition where training data is limited and new talkers, new words, new dialects, new phrases, and new environments are continuously encountered. Adaptation also provides a degree of robustness by compensating for minor variabilities in characteristics of processing elements. Traditional statistical techniques are not adaptive but typically process all training data simultaneously before being used with new data. Neural net classifiers are also non-parametric and make weaker assumptions concerning the shapes of underlying distributions than traditional statistical classifiers. They may thus prove to be more robust when distributions are generated by nonlinear processes and are strongly non-Gaussian. Designing artificial neural nets to solve

Reprinted from *IEEE ASSP Magazine*, April 1987, pp. 4–22.

problems and studying real biological nets may also change the way we think about problems and lead to new insights and algorithmic improvements.

Work on artificial neural net models has a long history. Development of detailed mathematical models began more than 40 years ago with the work of McCulloch and Pitts [30], Hebb [17], Rosenblatt [39], Widrow [47] and others [38]. More recent work by Hopfield [18, 19, 20], Rumelhart and McClelland [40], Sejnowski [43], Feldman [9], Grossberg [15], and others has led to a new resurgence of the field. This new interest is due to the development of new net topologies and algorithms [18, 19, 20, 41, 9], new analog VLSI implementation techniques [31], and some intriguing demonstrations [43, 20] as well as by a growing fascination with the functioning of the human brain. Recent interest is also driven by the realization that human-like performance in the areas of speech and image recognition will require enormous amounts of processing. Neural nets provide one technique for obtaining the required processing capacity using large numbers of simple processing elements operating in parallel.

This paper provides an introduction to the field of neural nets by reviewing six important neural net models that can be used for pattern classification. These massively parallel nets are important building blocks which can be used to construct more complex systems. The main purpose of this review is to describe the purpose and design of each net in detail, to relate each net to existing pattern classification and clustering algorithms that are normally implemented on sequential von Neumann computers, and to illustrate design principles used to obtain parallelism using neural-like processing elements.

Neural net and traditional classifiers

Block diagrams of traditional and neural net classifiers are presented in Fig. 2. Both types of classifiers determine which of M classes is most representative of an unknown static input pattern containing N input elements. In a speech recognizer the inputs might be the output envelope values from a filter bank spectral analyzer sampled at one time instant and the classes might represent different vowels. In an image classifier the inputs might be the gray scale level of each pixel for a picture and the classes might represent different objects.

The traditional classifier in the top of Fig. 2 contains two stages. The first computes matching scores for each class and the second selects the class with the maximum score. Inputs to the first stage are symbols representing values of the N input elements. These symbols are entered sequentially and decoded from the external symbolic form into an internal representation useful for performing arithmetic and symbolic operations. An algorithm computes a matching score for each of the M classes which indicates how closely the input matches the exemplar pattern for each class. This exemplar pattern is that pattern which is most representative of each class. In many situations a probabilistic model is used to model the generation of input patterns from exemplars and the matching score represents the likelihood or probability that the input pattern was generated from each of the M possible exemplars. In those cases, strong assumptions are typically made concerning underlying distributions of the input elements. Parameters of distributions can then be estimated using a training data as shown in Fig. 2. Multivariate Gaussian distributions are often used leading to relatively simple algorithms for computing matching scores [7]. Matching scores are coded into symbolic representations and passed sequentially to the second stage of

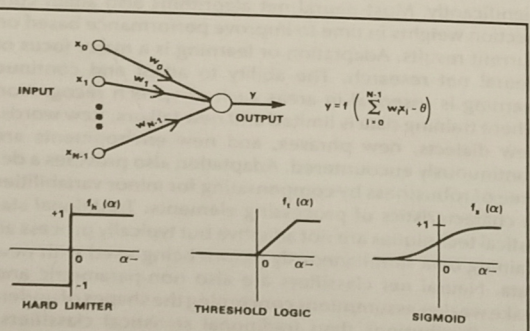


Figure 1. Computational element or node which forms a weighted sum of N inputs and passes the result through a nonlinearity. Three representative nonlinearities are shown.

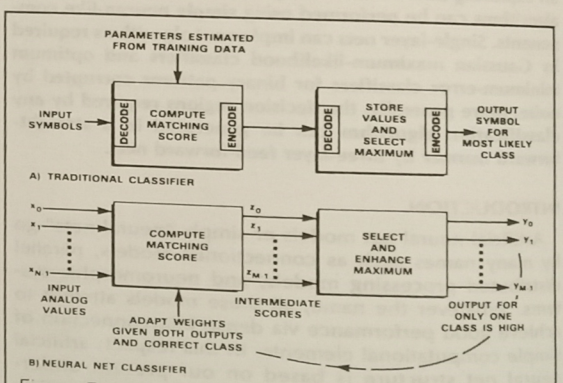


Figure 2. Block diagrams of traditional (A) and neural net (B) classifiers. Inputs and outputs of the traditional classifier are passed serially and internal computations are performed sequentially. In addition, parameters are typically estimated from training data and then held constant. Inputs and outputs to the neural net classifier are in parallel and internal computations are performed in parallel. Internal parameters or weights are typically adapted or trained during use using the output values and labels specifying the correct class.

the classifier. Here they are decoded and the class with the maximum score is selected. A symbol representing that class is then sent out to complete the classification task.

An adaptive neural net classifier is shown at the bottom of Fig. 2. Here input values are fed in parallel to the first stage via N input connections. Each connection carries an analog value which may take on two levels for binary inputs or may vary over a large range for continuous valued inputs. The first stage computes matching scores and outputs these scores in parallel to the next stage over M analog output lines. Here the maximum of these values is selected and enhanced. The second stage has one output for each of the M classes. After classification is complete, only that output corresponding to the most likely class will be on strongly or "high"; other outputs will be "low". Note that in this design, outputs exist for every class and that this multiplicity of outputs must be preserved in further processing stages as long as the classes are considered distinct. In the simplest classification system these output lines might go directly to lights with labels that specify class identities. In more complicated cases they may go to further stages of processing where inputs from other modalities or temporal dependencies are taken into consideration. If the correct class is provided, then this information and the classifier outputs can be fed back to the first stage of the classifier to adapt weights using a learning algorithm as shown in Fig. 2. Adaptation will make a correct response more likely for succeeding input patterns that are similar to the current pattern.

The parallel inputs required by neural net classifiers suggest that real-time hardware implementations should include special purpose pre-processors. One strategy for designing such processors is to build physiologically-based pre-processors modeled after human sensory systems. A pre-processor for image classification modeled after the retina and designed using analog VLSI circuitry is described in [31]. Pre-processor filter banks for speech recognition that are crude analogs of the cochlea have also been constructed [34, 29]. More recent physiologically-based pre-processor algorithms for speech recognition attempt to provide information similar to that available on the auditory nerve [11, 44, 27, 5]. Many of these algorithms include filter bank spectral analysis, automatic gain control, and processing which uses timing or synchrony information in addition to information from smoothed filter output envelopes.

Classifiers in Fig. 2 can perform three different tasks. First, as described above, they can identify which class best represents an input pattern, where it is assumed that inputs have been corrupted by noise or some other process. This is a classical decision theory problem. Second, the classifiers can be used as a content-addressable or associative memory, where the class exemplar is desired and the input pattern is used to determine which exemplar to produce. A content-addressable memory is useful when only part of an input pattern is available and the complete pattern is required, as in bibliographic retrieval of journal references from partial information. This normally requires the addition of a third stage in Fig. 2 to regenerate the exemplar for the most likely class. An additional stage is unnecessary for some neural nets such as the Hopfield net which are designed specifically as content-addressable memories. A third task these classifiers can perform is to vector quantize [28] or cluster [16, 7] the N inputs into M clusters. Vector quantizers are used in image and speech transmission systems to reduce the number of bits necessary to transmit analog data. In speech and image recognition applications they are used to compress the amount of data that must be processed without losing important information. In either application the number of clusters can be pre-specified or may be allowed to grow up to a limit determined by the number ber of nodes available in the first stage.

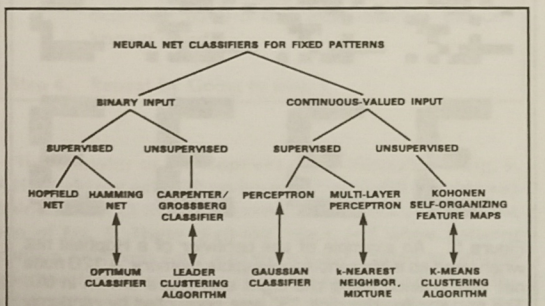


Figure 3. A taxonomy of six neural nets that can be used as classifiers. Classical algorithms which are most similar to the neural net models are listed along the bottom.

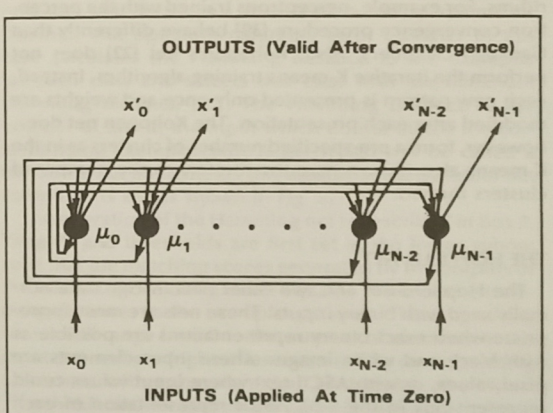


Figure 4. A Hopfield neural net that can be used as a content-addressable memory. An unknown binary input pattern is applied at time zero and the net then iterates until convergence when node outputs remain unchanged. The output is that pattern produced by node outputs after convergence.

A TAXONOMY OF NEURAL NETS

A taxonomy of six important neural nets that can be used for classification of static patterns is presented in Fig. 3. This taxonomy is first divided between nets with binary and continuous valued inputs. Below this, nets are divided between those trained with and without supervision. Nets trained with supervision such as the Hopfield net [18] and perceptrons [39] are used as associative memories or as classifiers. These nets are provided with side information or labels that specify the correct class for new input patterns during training. Most traditional statistical classifiers, such as Gaussian classifiers [7], are trained with supervision using labeled training data. Nets trained without supervision, such as the Kohonen's feature-map forming nets [22], are used as vector quantizers or to form clusters. No information concerning the correct class is provided to these nets during training. The classical K-means [7] and leader [16] clustering algorithms are trained without supervision. A further difference between nets, not indicated in Fig. 3, is whether adaptive training is supported. Although all the nets shown can be trained adaptively, the Hopfield net and the Hamming net are generally used with fixed weights.

The algorithms listed at the bottom of Fig. 3 are those classical algorithms which are most similar to or perform the same function as the corresponding neural net. In some cases a net implements a classical algorithm exactly. For example, the Hamming net [25] is a neural net implementation of the optimum classifier for binary patterns corrupted by random noise [10]. It can also be shown that the perceptron structure performs those calculations required by a Gaussian classifier [7] when weights and thresholds are selected appropriately. In other cases the neural net algorithms are different from the classical algorithms. For example, perceptrons trained with the perceptron convergence procedure [39] behave differently than Gaussian classifiers. Also, Kohonen's net [22] does not perform the iterative K-means training algorithm. Instead, each new pattern is presented only once and weights are modified after each presentation. The Kohonen net does, however, form a pre-specified number of clusters as in the K-means algorithm, where the K refers to the number of clusters formed.

THE HOPFIELD NET

The Hopfield net and two other nets in Fig. 3 are normally used with binary inputs. These nets are most appropriate when exact binary representations are possible as with black and white images where input elements are pixel values, or with ASCII text where input values could represent bits in the 8-bit ASCII representation of each character. These nets are less appropriate when input values are actually continuous, because a fundamental representation problem must be addressed to convert the analog quantities to binary values.

Hopfield rekindled interest in neural nets by his extensive work on different versions of the Hopfield net [18, 19, 20]. This net can be used as an associative memory or to solve optimization problems. One version of the original net [18] which can be used as a content addressable memory is described in this paper. This net, shown in Fig. 4, has N nodes containing hard limiting nonlinearities and binary inputs and outputs taking on the values +1 and −1. The output of each node is fed back to all other nodes via weights denoted t_{ij}. The operation of this net is described in Box 1. First, weights are set using the given recipe from exemplar patterns for all classes. Then an unknown pattern is imposed on the net at time zero by forcing the output of the net to match the unknown pattern. Following this initialization, the net iterates in discrete time steps using the given formula. The net is considered to have converged when outputs no longer change on successive iterations. The pattern specified by the node outputs after convergence is the net output.

Hopfield [18] and others [4] have proven that this net converges when the weights are symmetric ($t_{ij} = t_{ji}$) and

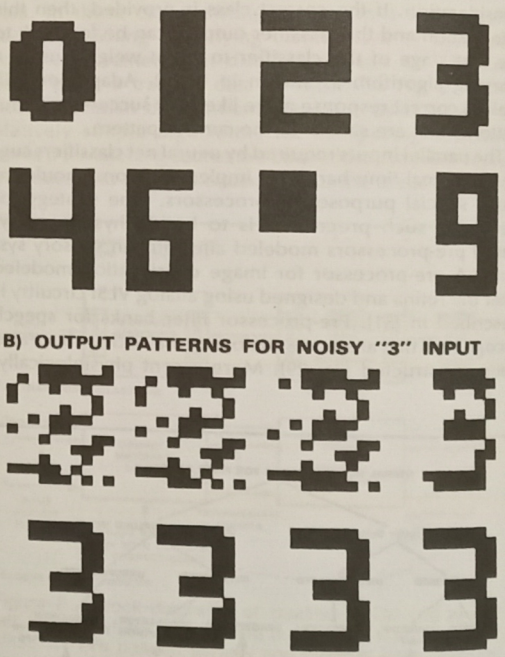


Figure 5. An example of the behavior of a Hopfield net when used as a content-addressable memory. A 120 node net was trained using the eight exemplars shown in (A). The pattern for the digit "3" was corrupted by randomly reversing each bit with a probability of .25, and then applied to the net at time zero. Outputs at time zero and after the first seven iterations are shown in (B).

node outputs are updated asynchronously using the equations in Box 1. Hopfield [19] also demonstrated that the net converges when graded nonlinearities similar to the sigmoid nonlinearity in Fig. 1 are used. When the Hopfield net is used as an associative memory, the net output after convergence is used directly as the complete restored memory. When the Hopfield net is used as a classifier, the output after convergence must be compared to the M exemplars to determine if it matches an exemplar exactly. If it does, the output is that class whose exemplar matched the output pattern. If it does not then a "no match" result occurs.

Box 1. Hopfield Net Algorithm

Step 1. Assign Connection Weights

$$t_{ij} = \begin{cases} \sum_{s=0}^{M-1} x_i^s x_j^s, & i \neq j \\ 0, & i = j,\ 0 \leq i, j \leq M\text{-}1 \end{cases}$$

In this Formula t_{ij} is the connection weight from node i to node j and x_i^s which can be +1 or −1 is element i of the exemplar for class s.

Step 2. Initialize with Unknown Input Pattern

$$\mu_i(0) = x_i, \qquad 0 \leq i \leq N - 1$$

In this Formula $\mu_i(t)$ is the output of node i at time t and x_i which can be +1 or −1 is element i of the input pattern.

Step 3. Iterate Until Convergence

$$\mu_j(t + 1) = f_h\left[\sum_{i=0}^{N-1} t_{ij}\,\mu_i(t)\right], \quad 0 \leq j \leq M\text{-}1$$

The function f_h is the hard limiting nonlinearity from Fig. 1. The process is repeated until node outputs remain unchanged with further iterations. The node outputs then represent the exemplar pattern that best matches the unknown input.

Step 4. Repeat by Going to Step 2

The behavior of the Hopfield net is illustrated in Fig. 5. A Hopfield net with 120 nodes and thus 14,400 weights was trained to recall the eight exemplar patterns shown at the top of Fig. 5. These digit-like black and white patterns contain 120 pixels each and were hand crafted to provide good performance. Input elements to the net take on the value +1 for black pixels and −1 for white pixels. In the example presented, the pattern for the digit "3" was corrupted by randomly reversing each bit independently from +1 to −1 and vice versa with a probability of 0.25. This pattern was then applied to the net at time zero. Patterns produced at the output of the net on iterations zero to seven are presented at the bottom of Fig. 5. The corrupted input pattern is present unaltered at iteration zero. As the net iterates the output becomes more and more like the correct exemplar pattern until at iteration six the net has converged to the pattern for the digit three.

The Hopfield net has two major limitations when used as a content addressable memory. First, the number of patterns that can be stored and accurately recalled is severely limited. If too many patterns are stored, the net may converge to a novel spurious pattern different from all exemplar patterns. Such a spurious pattern will produce a "no match" output when the net is used as a classifier. Hopfield [18] showed that this occurs infrequently when exemplar patterns are generated randomly and the number of classes (M) is less than .15 times the number of input elements or nodes in the net (N). The number of classes is thus typically kept well below .15N. For example, a Hopfield net for only 10 classes might require more than 70 nodes and more than roughly 5,000 connection weights. A second limitation of the Hopfield net is that an exemplar pattern will be unstable if it shares many bits in common with another exemplar pattern. Here an exemplar is considered unstable if it is applied at time zero and the net converges to some other exemplar. This problem can be eliminated and performance can be improved by a number of orthogonalization procedures [14, 46].

THE HAMMING NET

The Hopfield net is often tested on problems where inputs are generated by selecting an exemplar and reversing bit values randomly and independently with a given probability [18, 12, 46]. This is a classic problem in communications theory that occurs when binary fixed-length signals are sent through a memoryless binary symmetric channel. The optimum minimum error classifier in this case calculates the Hamming distance to the exemplar for each class and selects that class with the minimum Hamming distance [10]. The Hamming distance is the number of bits in the input which do not match the corresponding exemplar bits. A net which will be called a Hamming net implements this algorithm using neural net components and is shown in Fig. 6.

The operation of the Hamming net is described in Box 2. Weights and thresholds are first set in the lower subnet such that the matching scores generated by the outputs of the middle nodes of Fig. 6 are equal to N minus the Hamming distances to the exemplar patterns. These matching scores will range from 0 to the number of elements in the input (N) and are highest for those nodes corresponding to classes with exemplars that best match the input. Thresholds and weights in the MAXNET subnet are fixed. All thresholds are set to zero and weights from each node to itself are 1. Weights between nodes are inhibitory with a value of $-\epsilon$ where $\epsilon < 1/M$.

After weights and thresholds have been set, a binary pattern with N elements is presented at the bottom of the Hamming net. It must be presented long enough to allow

the matching score outputs of the lower subnet to settle and initialize the output values of the MAXNET. The input is then removed and the MAXNET iterates until the output of only one node is positive. Classification is then complete and the selected class is that corresponding to the node with a positive output.

The behavior of the Hamming net is illustrated in Fig. 7.

Box 2. Hamming Net Algorithm

Step 1. Assign Connection Weights and Offsets

In the lower subnet:

$$w_{ij} = \frac{x_i^j}{2}, \qquad \theta_j = \frac{N}{2},$$
$$0 \le i \le N - 1, \qquad 0 \le j \le M - 1$$

In the upper subnet:

$$t_{kl} = \begin{cases} 1, & k = l \\ -\varepsilon, & k \ne l, \end{cases} \qquad \varepsilon < \frac{1}{M},$$
$$0 \le k, l \le M - 1$$

In these equations w_{ij} is the connection weight from input i to node j in the lower subnet and θ is the threshold in that node. The connection weight from node k to node l in the upper subnet is t_{kl} and all thresholds in this subnet are zero. x_i^j is element i of exemplar j as in Box 1.

Step 2. Initialize with Unknown Input Pattern

$$\mu_j(0) = f_t\left(\sum_{i=0}^{N-1} w_{ij}x_i - \theta_j\right)$$
$$0 \le j \le M - 1$$

In this equation $\mu_j(t)$ is the output of node j in the upper subnet at time t, x_i is element i of the input as in Box 1, and f_t is the threshold logic nonlinearity from Fig. 1. Here and below it is assumed that the maximum input to this nonlinearity never causes the output to saturate.

Step 3. Iterate Until Convergence

$$\mu_j(t + 1) = f_t\left(\mu_j(t) - \varepsilon\sum_{k \ne j} \mu_k(t)\right)$$
$$0 \le j, k \le M - 1$$

This process is repeated until convergence after which the output of only one node remains positive.

Step 4. Repeat by Going to Step 2

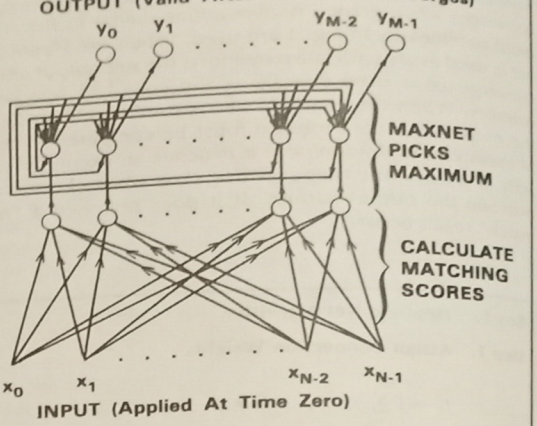


Figure 6. A feed-forward Hamming net maximum likelihood classifier for binary inputs corrupted by noise. The lower subnet calculates N minus the Hamming distance to M exemplar patterns. The upper net selects that node with the maximum output. All nodes use threshold-logic nonlinearities where it is assumed that the outputs of these nonlinearities never saturate.

The four plots in this figure show the outputs of nodes in a MAXNET with 100 nodes on iterations 0, 3, 6, and 9. These simulations were obtained using randomly selected exemplar patterns with 1000 elements each. The exemplar for class 50 was presented at time zero and then removed. The matching score at time zero is maximum (1000) for node 50 and has a random value near 500 for other nodes. After only 3 iterations, the outputs of all nodes except node 50 have been greatly reduced and after 9 iterations only the output for node 50 is non-zero. Simulations with different probabilities of reversing bits on input patterns and with different numbers of classes and elements in the input patterns have demonstrated that the MAXNET typically converges in less than 10 iterations in this application [25]. In addition, it can be proven that the MAXNET will always converge and find the node with the maximum value when $\varepsilon < 1/M$ [25].

The Hamming net has a number of obvious advantages over the Hopfield net. It implements the optimum minimum error classifier when bit errors are random and independent, and thus the performance of the Hopfield net must either be worse than or equivalent to that of the Hamming net in such situations. Comparisons between the two nets on problems such as character recognition, recognition of random patterns, and bibliographic retrieval have demonstrated this difference in performance [25]. The Hamming net also requires many fewer connections than the Hopfield net. For example, with 100 inputs and 10 classes the Hamming net requires

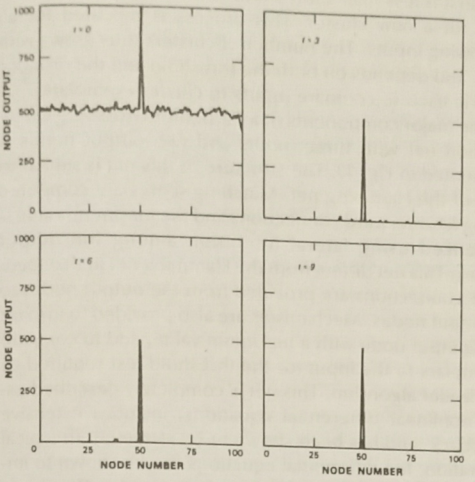


Figure 7. Node outputs for a Hamming net with 1,000 binary inputs and 100 output nodes or classes. Output values of all 100 nodes are presented at time zero and after 3, 6, and 9 iterations. The input was the exemplar pattern corresponding to output node 50.

only 1,100 connections while the Hopfield net requires almost 10,000. Furthermore, the difference in number of connections required increases as the number of inputs increases, because the number of connections in the Hopfield net grows as the square of the number of inputs while the number of connections in the Hamming net grows linearly. The Hamming net can also be modified to be a minimum error classifier when errors are generated by reversing input elements from +1 to −1 and from −1 to +1 asymmetrically with different probabilities [25] and when the values of specific input elements are unknown [2]. Finally, the Hamming net does not suffer from spurious output patterns which can produce a "no-match" result.

SELECTING OR ENHANCING THE MAXIMUM INPUT

The need to select or enhance the input with a maximum value occurs frequently in classification problems. Several different neural nets can perform this operation. The MAXNET described above uses heavy lateral inhibition similar to that used in other net designs where a maximum was desired [20, 22, 9]. These designs create a "winner-take-all" type of net whose design mimics the heavy use of lateral inhibition evident in the biological neural nets of the human brain [21]. Other techniques to pick a maximum are also possible [25]. One is illustrated in Fig. 8. This figure shows a comparator subnet which is described in [29]. It uses threshold logic nodes to pick the maximum of two inputs and then feeds this maximum value forward. This net is useful when the maximum value must be passed unaltered to the output. Comparator subnets can be layered into roughly $\log_2(M)$ layers to pick the maximum of M inputs. A net that uses these subnets to pick the maximum of 8 inputs is presented in Fig. 9.

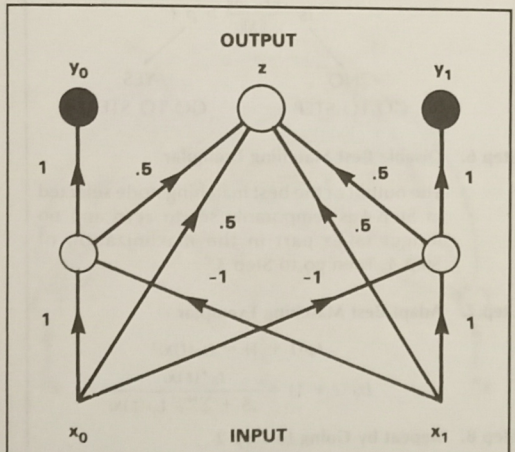


Figure 8. A comparator subnet that selects the maximum of two analog inputs. The output labeled z is the maximum value and the outputs labeled y_0 and y_1 indicate which input was maximum. Internal thresholds on threshold logic nodes (open circles) and hard limiting nodes (filled circles) are zero. Weights are as shown.

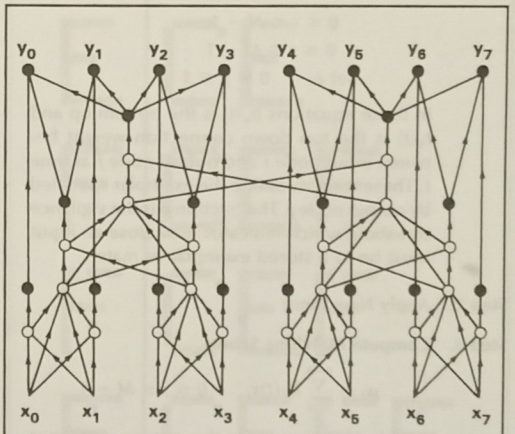


Figure 9. A feed-forward net that determines which of eight inputs is maximum using a binary tree and comparator subnets from Fig. 8. After an input vector is applied, only that output corresponding to the maximum input element will be high. Internal thresholds on threshold-logic nodes (open circles) and on hard limiting nodes (filled circles) are zero except for the output nodes. Thresholds in the output nodes are 2.5. Weights for the comparator subnets are as in Fig. 8 and all other weights are 1.